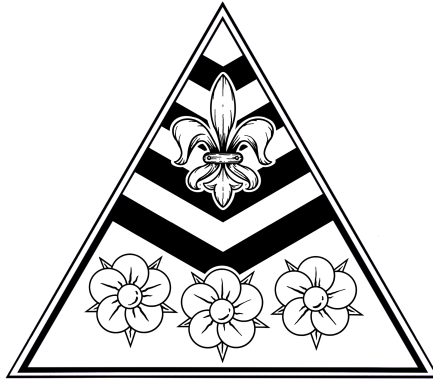
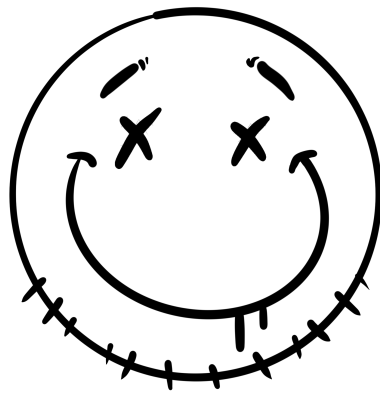




Lab 13: Log Analysis with Splunk

Keith Schoen

CSEC3755-001

Dr. Daniel Pittman

12 May 2026

Contents

1	Standing Up Splunk	1
1.1	Running Splunk	1
1.2	Setting Sysmon Index in Splunk	1
1.3	Sysmon field-extractions (<code>props.conf</code>)	2
2	Forwarder Setup	3
2.1	Forwarder Configuration	3
2.2	Forwarder Results	4
3	ART & SPL	5
3.1	ART Setup	5
3.2	SPL Queries	5
4	The Dashboard	7

1 Standing Up Splunk

The following section contains the steps taken to ensure that Splunk was installed and running successfully. Proofs are given in screenshot and written formats.

1.1 Running Splunk

```
csec3755@ubuntu:~$ docker run --name splunk --hostname splunk -p 8000:8000 -p 9997:9997 -e "SPLUNK_PASSWORD=ChangeMe123!" -e "SPLUNK_START_ARGS=--accept-license" -e "SPLUNK_GENERAL_TERMS=--accept-sgt-current-at-splunk-com" -d splunk/splunk:latest
```

Figure 1: Screenshot of `docker run` command ran on Ubuntu.

```
NAME                  NAMES
38f7e938190d         splunk/splunk:latest           "/sbin/entrypoint.sh..."
7 minutes ago       Up 7 minutes (healthy)        8065/tcp, 8088-8089/tcp,
8191/tcp, 0.0.0.0:8000->8000/tcp, [::]:8000->8000/tcp, 0.0.0.0:9997->9997/tcp,
[::]:9997->9997/tcp, 9887/tcp
splunk
8e2840b5c23f         ghcr.io/msu-denver/medtech-patient-portal:latest "python app.py"
3 weeks ago        Up About an hour             0.0.0.0:80->5000/tcp, [::]:80->5000/tcp
```

Figure 2: Screenshot of `docker ps` results showing the Splunk container is healthy.

Ubuntu's IP Address: 127.0.0.1

1.2 Setting Sysmon Index in Splunk

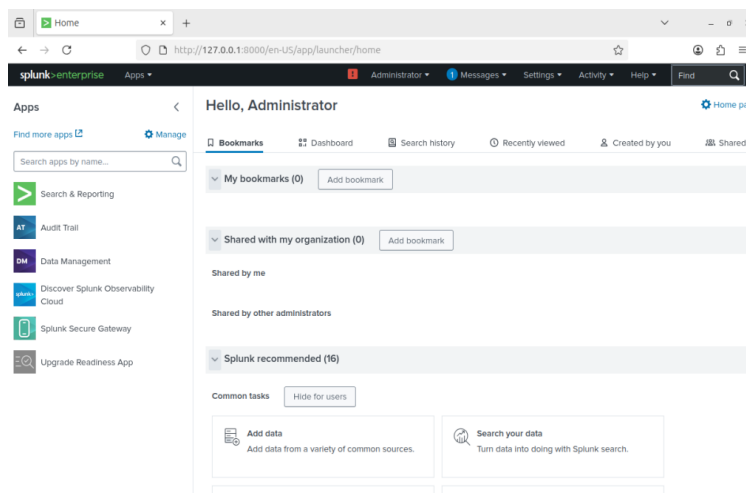


Figure 3: Screenshot of Splunk GUI homepage accessed as Administrator.

Name	Status	Type	App	Current Size	Max Size	Event Count	Earliest Event	Latest Event	Home Path	Frozen Path
telemetry	Active	Events	system	1 MB	488.28 GB	0	—	—	\$SPLUNK_DB/telemetry/db	—
thefishbucket	Active	Events	system	1 MB	488.28 GB	0	—	—	\$SPLUNK_DB/thefishbucket/db	—
history	Active	Events	system	1 MB	488.28 GB	0	—	—	\$SPLUNK_DB/history/db	—
main	Active	Events	system	1 MB	488.28 GB	0	—	—	\$SPLUNK_DB/defaultdb	—
splunklogger	Inactive	Events	system	0 B	488.28 GB	0	—	—	\$SPLUNK_DB/splunklogger/db	—
summary	Active	Events	system	1 MB	488.28 GB	0	—	—	\$SPLUNK_DB/summarydb	—
sysmon	Active	Events	search	1 MB	500 GB	0	—	—	\$SPLUNK_DB/sysmon/db	—

Figure 4: Screenshot of updated Splunk GUI index page showing 'sysmon' index.

Why a sysmon index? Creating a dedicated Sysmon index keeps your endpoint telemetry isolated from other log sources, making your SPL queries faster and more precise since you can scope searches to `index=sysmon` instead of sifting through everything in main.

1.3 Sysmon field-extractions (props.conf)

```
csec3755@ubuntu: ~
EXTRACT-Image = <Data Name='Image'>(?(Image)[^<]+)
EXTRACT-CommandLine = <Data Name='CommandLine'>(?(CommandLine)[^<]+)
EXTRACT-ParentImage = <Data Name='ParentImage'>(?(ParentImage)[^<]+)
EXTRACT-ParentCommandLine = <Data Name='ParentCommandLine'>(?(ParentCommandLine)[^<]+)
EXTRACT-User = <Data Name='User'>(?(User)[^<]+)
EXTRACT-TargetFilename = <Data Name='TargetFilename'>(?(TargetFilename)[^<]+)
EXTRACT-TargetObject = <Data Name='TargetObject'>(?(TargetObject)[^<]+)
EXTRACT-Details = <Data Name='Details'>(?(Details)[^<]+)
EXTRACT-PipeName = <Data Name='PipeName'>(?(PipeName)[^<]+)
EXTRACT-ProcessId = <Data Name='ProcessId'>(?(ProcessId)[^<]+)
EXTRACT-Hashes = <Data Name='Hashes'>(?(Hashes)[^<]+)
EXTRACT-TargetImage = <Data Name='TargetImage'>(?(TargetImage)[^<]+)
EXTRACT-SourceImage = <Data Name='SourceImage'>(?(SourceImage)[^<]+)
EXTRACT-GrantedAccess = <Data Name='GrantedAccess'>(?(GrantedAccess)[^<]+)
EXTRACT-CallTrace = <Data Name='CallTrace'>(?(CallTrace)[^<]+)
EXTRACT-DestinationIp = <Data Name='DestinationIp'>(?(DestinationIp)[^<]+)
EXTRACT-DestinationPort = <Data Name='DestinationPort'>(?(DestinationPort)[^<]+)

sudo docker restart splunk-props.conf splunk:/opt/splunk/etc/system/local/props.conf
[sudo] password for csec3755:
Successfully copied 3.07kB to splunk:/opt/splunk/etc/system/local/props.conf
splunk
csec3755@ubuntu:~$
```

Figure 5: Screenshot of confirmation of successful props.conf extraction.

2 Forwarder Setup

The following section details the setup and configuration of the Splunk forwarder to collect and forward logs from Windows and Sysmon sources. Proofs are given in screenshot and written formats.

2.1 Forwarder Configuration

```
PS C:\WINDOWS\system32> msixexec.exe /i C:\Users\Public\Downloads\splunkforwarder.msi AGREETOLICENSE=Yes SPLUNKPASSWORD=c
hangeme RECEIVING_INDEXER="127.0.0.1:9997" WINEVENTLOG_SEC_ENABLE=1 WINEVENTLOG_SYS_ENABLE=1 WINEVENTLOG_APP_ENABLE=1 LA
UNCHSPLUNK=1 SERVICESTARTTYPE=auto /quiet
```

Figure 6: `msiexec` command ran to install the Splunk forwarder.

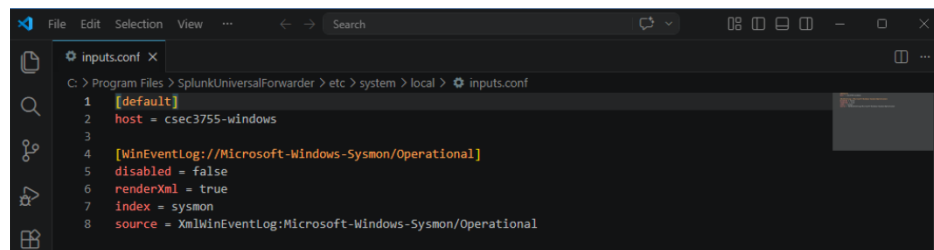


Figure 7: Contents of the `inputs.conf` file.

```

NAMES
38f7e938190d splunk/splunk:latest "/sbin/entrypo
int.sh..." 7 minutes ago Up 7 minutes (healthy) 8065/tcp, 8088-8089/tcp,
8191/tcp, 0.0.0.0:8000->8000/tcp, [::]:8000->8000/tcp, 0.0.0.0:9997->9997/tcp,
[::]:9997->9997/tcp, 9887/tcp
splunk
8e2840b5c23f ghcr.io/msu-denver/medtech-patient-portal:latest "python app.py
" 3 weeks ago Up About an hour 0.0.0.0:80->5000/tcp, [::]:80->5000/tcp
```

Figure 8: Output of the `docker ps` command showing the Splunk forwarder is running.

2.2 Forwarder Results

New Search

index=symon EventCode=1 | table _time, User, Image, CommandLine, ParentImage | head 20

1,036 events (5/11/26 7:00:00.000 PM to 5/12/26 7:27:44.000 PM) No Event Sampling

Events Patterns Statistics (20) Visualization

Show: 20 Per Page Format Preview On

_time	User	Image	CommandLine	ParentImage
2026-05-12 18:19:34.050	NT AUTHORITY\SYSTEM	C:\Windows\Microsoft.NET\Framework64\v4.0.30319\mscorsvw.exe	C:\Windows\Microsoft.NET\Framework64\v4.0.30319\mscorsvw.exe -StartupEvent 420 -InterruptEvent 0 -NGENProcess 440 -Pipe 430 -Comment "NGen Worker Process"	C:\Windows\Microsoft.NET\Framework64\v4.0.30319\mscorsvw.exe
2026-05-12 18:19:33.994	NT AUTHORITY\SYSTEM	C:\Windows\Microsoft.NET\Framework64\v4.0.30319\mscorsvw.exe	C:\Windows\Microsoft.NET\Framework64\v4.0.30319\mscorsvw.exe -StartupEvent 488 -InterruptEvent 0 -NGENProcess 490 -Pipe 474 -Comment "NGen Worker Process"	C:\Windows\Microsoft.NET\Framework64\v4.0.30319\mscorsvw.exe
2026-05-12 18:19:33.926	NT AUTHORITY\SYSTEM	C:\Windows\Microsoft.NET\Framework64\v4.0.30319\mscorsvw.exe	C:\Windows\Microsoft.NET\Framework64\v4.0.30319\mscorsvw.exe -StartupEvent 430 -InterruptEvent 0 -NGENProcess 414 -Pipe 450 -Comment "NGen Worker Process"	C:\Windows\Microsoft.NET\Framework64\v4.0.30319\mscorsvw.exe
2026-05-12 18:19:33.864	NT AUTHORITY\SYSTEM	C:\Windows\Microsoft.NET\Framework64\v4.0.30319\mscorsvw.exe	C:\Windows\Microsoft.NET\Framework64\v4.0.30319\mscorsvw.exe -StartupEvent 420 -InterruptEvent 0 -NGENProcess 450 -Pipe 430 -Comment "NGen Worker Process"	C:\Windows\Microsoft.NET\Framework64\v4.0.30319\mscorsvw.exe

Figure 9: Splunk results for Event Code 1.

Why use a forwarder?

There are many reasons why you might use a forwarder instead of pulling directly from a machine such as architecture limitations, efficiency and reliability, but the primary reason is scalability. In an enterprise environment, there will be dozens or endpoints. To install and check each on individually would be a time-consuming nightmare. Instead, we have a central server that each machine will forward log to. From the central server, we can manage and analyze all the logs in one place.

3 ART & SPL

In this section, we will generate events with Atomic Red Team and write SPL queries to analyze them.

3.1 ART Setup

```
PS C:\WINDOWS\system32> Invoke-AtomicTest T1059.001 -TestNumbers 17 -ShowDetails
PathToAtomicsFolder = C:\AtomicRedTeam\atomics

[*****BEGIN TEST*****]
Technique: Command and Scripting Interpreter: PowerShell T1059.001
Atomic Test Name: PowerShell Command Execution
Atomic Test Number: 17
Atomic Test GUID: a538de64-1c74-46ed-aa60-b995ed302598
Description: Use of obfuscated PowerShell to execute an arbitrary command; outputs "Hello, from PowerShell!". Example is
from the 2021 Threat Detection Report by Red Canary.

Attack Commands:
Executor: command_prompt
ElevationRequired: False
Command:
powershell.exe -e #[obfuscated_code]
Command (with inputs):
powershell.exe -e jgAgACgAZwBjAG0AIAAoACCaaQBIAHsAMAB9ACCIAATAGYAIAnAHgAJwApACKAIAAoACTIAVwByACTIAKwAIAGKAdAAIACsAIgBIA
C0ASAAIACsAIgBVAHwMADAAGCCASAAIACsAIgBIAgWAIGArACIAbABVACwAIABmAHIAIGArACIAbWBTACAAUAAIACsAIgBVAHCAIGArACIAZQByAFMAIGArA
CIAAAIACsAIgBIAgWAbAAHACCIAgApAA==
[*****END TEST*****]
```

Figure 10: Confirmation of Atomic Red Team installation.

For my test, I selected T1016 - Network Configuration Discovery. I picked this technique because it is a common tactic used by adversaries to gather information about the network environment and because it seemed interesting and relevant to the goals of this lab.

3.2 SPL Queries

Query 1:

Search Query: `Index=sysmon EventCode!= Image=*powershell.exe* CommandLine=*-e* | table _time, User, Image, CommandLine, PartentImage`

Time range: Last 24 hours

1 event (5/11/26 10:00:00.000 PM to 5/12/26 10:56:36.000 PM) No Event Sampling

Events	Patterns	Statistics (1)	Visualization
Show: 20 Per Page	Format	Preview: On	
_time	User	Image	CommandLine
2026-05-12 20:41:54.030	CSEC3755-WIN\user	C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe	powershell.exe -e jgAgACgAZwBjAG0AIAAoACCaaQBIAHsAMAB9ACCIAATAGYAIAnAHgAJwApACKAIAAoACTIAVwByACTIAKwAIAGKAdAAIACsAIgBIA C0ASAAIACsAIgBVAHwMADAAGCCASAAIACsAIgBIAgWAIGArACIAbABVACwAIABmAHIAIGArACIAbWBTACAAUAAIACsAIgBVAHCAIGArACIAZQByAFMAIGArA CIAAAIACsAIgBIAgWAbAAHACCIAgApAA==

Figure 11: SPL query 1

This query searches for encoded PowerShell commands. T1016 uses commands that poke at a system to find configuration information but does not use any encoded inputs. The query result shown is most an artifact from the T1059.001 test ran earlier.

Query 2:

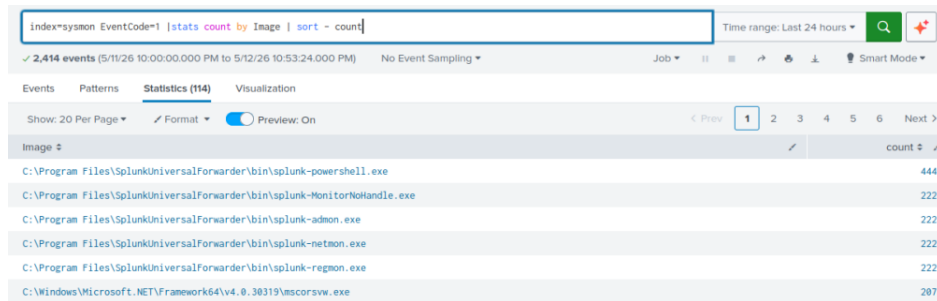


Figure 12: SPL query 2

This query checks for process frequency to aid in identifying anomalous behavior. Searching through the final pages of the results (where we start seeing single occurrences of processes) we can see our anomalous powershell.exe process.

Query 3:

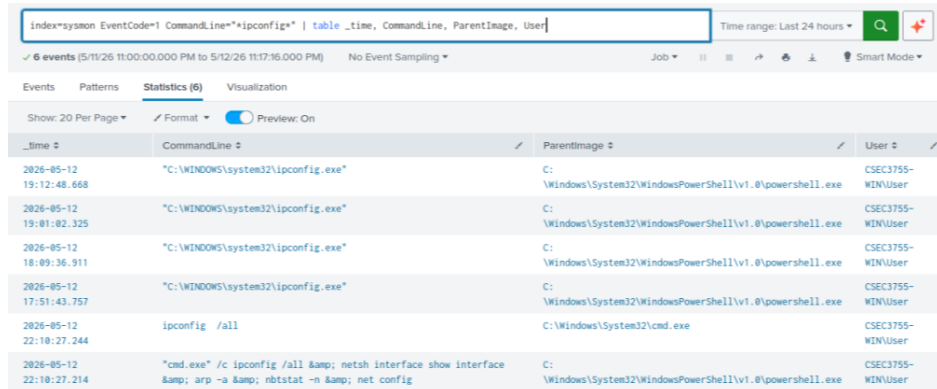


Figure 13: SPL query 3

This query is designed with the T1016 technique specifically in mind. Because we know this attack will attempt to gather network information, `ipconfig` is the most relevant command to look for and because it is not a command commonly used by people outside of IT, the query would return minimal results. The top four results are from myself generating traffic to test forwarding. Our last two results are clearly from our ART test; the two results coinciding with each other. The final result shows the `cmd.exe` file being executed through PowerShell, which includes `ipconfig/all` from the result before.

4 The Dashboard

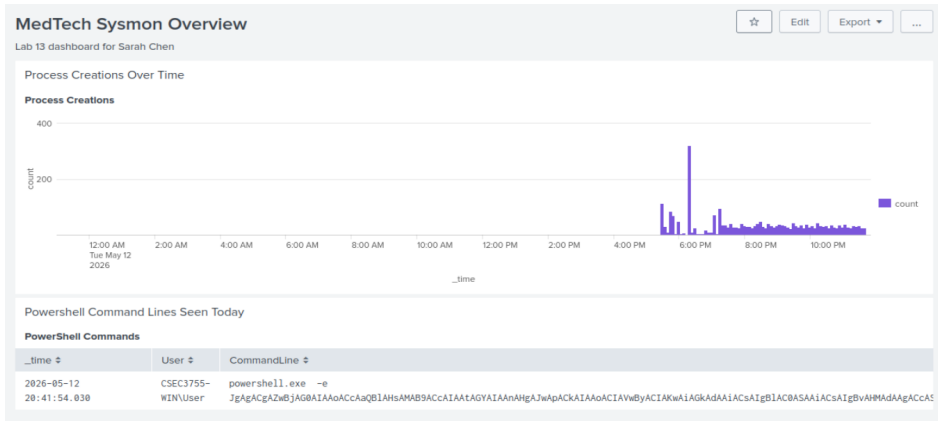


Figure 14: Splunk detection dashboard.

I added a panel to watch `powershell.exe` processes. This will help to detect general anomalous behavior. Initially, I used the SPL that was posted for this in the Lab, but with all of the normal Splunk traffice, the report was extremely messy. To counter, I added a filter to exclude Splunk standard events. Specifically, an analysis will be able to see the tell tale signs of an attacker possibly living off of land by seeing unusual patterns of execution, spikes in process frequency, and odd hours of operation. The next panel I would add would be to detect for network connections by destination to track for outside communications and possible C2 activity.

++report written with (minimal) help from Claude (Sonnet 4.6)++